

Reproducing Table 1 (S2C: HKU-IS → COD10K) — "Ours" rows

Everything needed to retrain the **Ours** (CSRDA) rows of Table 1 from a clean state, verified against the code actually in this repo ([MyTrain.py](#), [CLS.py](#), [Src/](#), [Eval/](#)) and against the machine this doc was written on.

Status of this checkout: branch [experiments/wip1](#), staged-but-uncommitted [.gitignore](#), [pyproject.toml](#), [uv.lock](#). [MyTrain.py](#) / [CLS.py](#) / [Src/](#) / [Eval/](#) are the upstream originals — unmodified. Reproduction below assumes they stay unmodified.

1. Target numbers

Table 1, S2C task (source = synthetic HKU-IS, target = real COD10K-train unlabeled, eval = COD10K-test).

Model	Setting	S α $\uparrow$	F β w $\uparrow$	E ϕ ad $\uparrow$	E ϕ m n $\uparrow$	E ϕ m x $\uparrow$	F β ad $\uparrow$	F β mn $\uparrow$	F β m x $\uparrow$	M $\downarrow$
SINet	Source-Only	0.6418	0.3707	0.7323	0.6570	0.7211	0.4653	0.4284	0.4718	0.0905
SINet	Mean Teacher	0.6984	0.4165	0.7092	0.7177	0.7805	0.4888	0.5106	0.5598	0.0915
SINet	Ours	0.7136	0.4814	0.7676	0.7443	0.7950	0.5548	0.5572	0.5960	0.0717
SINet-v2	Source-Only	0.6466	0.4121	0.7332	0.6986	0.7035	0.4736	0.4640	0.4697	0.0976

The column names map 1:1 onto [Eval/MyEval.py](#) output columns: [Smeasure](#), [wFmeasure](#), [adpEm](#), [meanEm](#), [maxEm](#), [adpFm](#), [meanFm](#), [maxFm](#), [MAE](#).

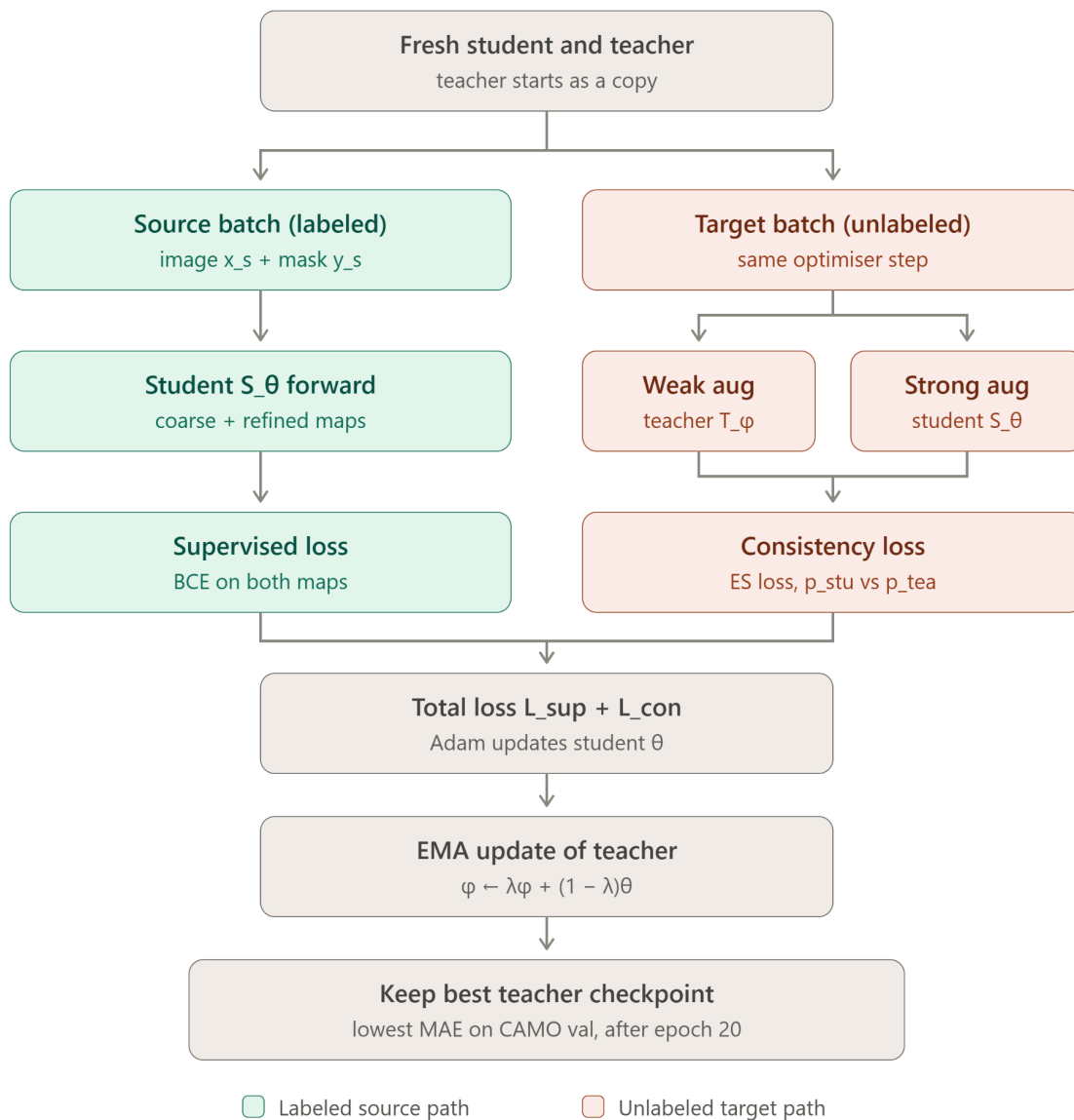
Paper hyperparameters for $\hat{S}2C$ — **already hard-coded** as a task override in [MyTrain.py:151-159](#), so you do **not** pass them on the CLI:

Symbol	Meaning	Value	Flag
λ	teacher EMA smoothing	0.996	<code>--alpha</code>
μ	CLS discrepancy multiplier	0.8	<code>--u</code>
τ	CLS confidence threshold	0.4	<code>--tau</code>
a, b, c	ESLoss edge / region / saliency-offset weights	0.9, 0.3, 0.5	<code>--a --b --c</code>
—	source root	<code>./Dataset/Source/HKU-IS/</code>	<code>--source_root</code>

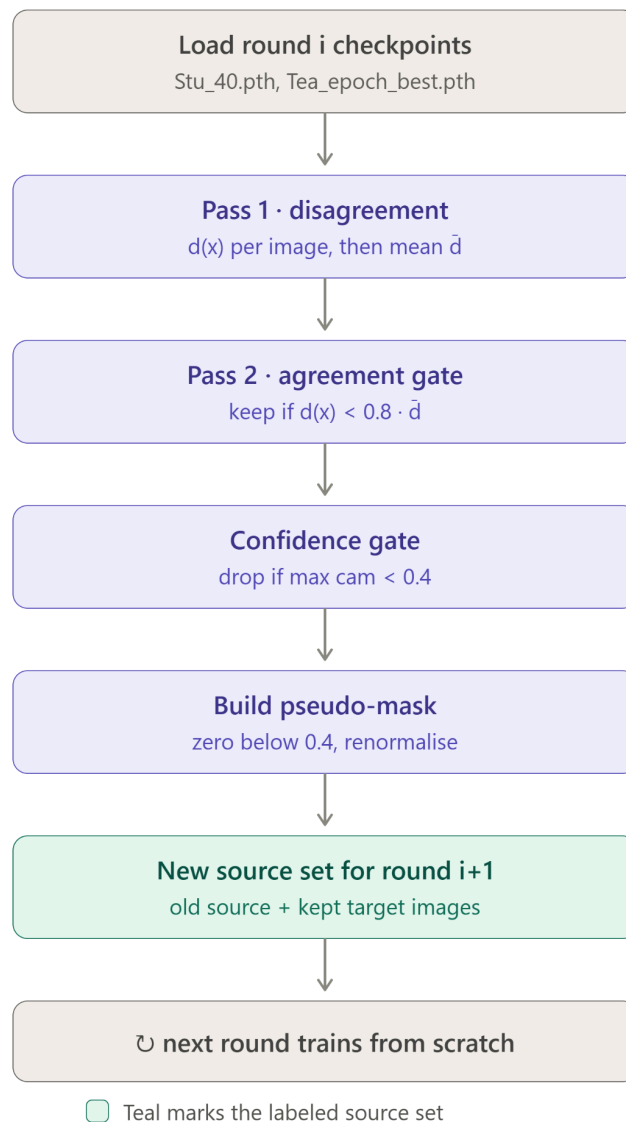
Optimizer/schedule (parser defaults, [MyTrain.py:127-135](#)): Adam, lr 1e-4, batch 16, trainsize 352, 40 epochs, decay $\times 0.1$ at epoch 30.

2. How the CSRDA framework works

CSRDA is a **cyclic** synthetic-to-real domain adaptation loop. One invocation of `MyTrain.py` runs `--iteration` rounds (default **2**). Each round is a full mean-teacher training run; between rounds, CLS harvests confident pseudo-labels from the real target domain and folds them into the source set for the next round.



Once a round finishes, the best teacher is used to mine pseudo-labels from the target set — but only for the images it is confident and consistent about.



The three mechanisms, precisely

(a) Mean-teacher consistency across the domain gap. Weak/strong augmentation split lives in [Src/utils/Dataloader.py:82-94](#): the *weak* view is resize+normalize only; the *strong* view adds `RandomAutocontrast` and `GaussianBlur(3,  $\sigma \in [0.1, 1.5]$ )`. Both views come from the same image (with a shared random horizontal flip applied first). The teacher sees the easy view and supervises the student on the hard view. Teacher weights are an EMA of the student with $\lambda=0.996$ ([Src/utils/tool.py:41-43](#)).

(b) ESLoss — the edge-aware + saliency-weighted consistency loss.
[Src/utils/tool.py:45-77](#):

$\text{edge_loss} = L1(|\nabla_{\text{sobel}} p_{\text{stu}}|, |\nabla_{\text{sobel}} p_{\text{tea}}|)$

$\text{region_loss} = \text{BCE}(p_{\text{stu}}, p_{\text{tea}}, \text{weight} = p_{\text{tea}} + c)$ # $c = 0.5$

$\text{ESLoss} = a \cdot \text{edge_loss} + b \cdot \text{region_loss}$ # $a = 0.9, b = 0.3$

The Sobel term is what makes it *camouflage-aware*: boundary agreement is weighted more heavily than area agreement, which is where synthetic→real transfer degrades most. The saliency weight $p_{\text{tea}} + c$ upweights pixels the teacher already believes are object.

(c) CLS — Confident Label Selection. A pseudo-label is admitted only if it passes **both** gates:

- *agreement gate* (μ): the student and teacher must disagree less than $0.8\times$ the dataset-average disagreement. This is a relative, self-calibrating threshold — no absolute confidence cutoff needed.
- *saturation gate* (τ): the teacher's map must actually contain a confident object ($\max(\text{cam}) \geq 0.4$), and everything below 0.4 is zeroed before renormalizing, so the pseudo-GT is a clean soft mask rather than a diffuse blob.

Two instances of ESLoss exist for a reason ([MyTrain.py:193-194](#)): training uses the *saliency-weighted* variant; CLS scoring uses `use_weighted_bce=False` so the ranking is not biased by how much object area a given teacher map happens to predict.

(d) The cycle. Round 2 does **not** fine-tune round 1 — student and teacher are re-instantiated from ImageNet weights at [MyTrain.py:170-187](#). What carries over is only the *data*: the source set is now synthetic HKU-IS **plus** the confidently-pseudo-labeled real COD10K images. That is the whole point — the supervised branch now contains real-domain samples, so `Lsup` itself pulls the model toward the target domain instead of only `Lcon` doing it.

3. Training data

What is present right now

Role	Path	Contents	State
Source (synthetic)	<code>Dataset/Source/HKU-IS/</code>	4447 <code>Image/*.jpg</code> + 4447 <code>GT/*.png</code>	✅ ready
Source (alt, C2C)	<code>Dataset/Source/COD10K/</code>	4446 + 4446	✅ present, not used for $\hat{S}2C$

⚠ **The archives nest their own top-level folder.** Extract into `Dataset/`, not into `Dataset/Target/` — otherwise you get `Dataset/Target/Target/Image/` and the loaders will fail.

Notes on the data pipeline

- `SrcDataset` pairs images↔GTs by **sorted filename order**, not by basename lookup ([Dataloader.py:14-18](#)). This is safe here: HKU-IS uses zero-padded numeric names (`0004.jpg` / `0004.png`) and CLS-generated files use `camourflage_*.png` for both image and GT, so the two sorted lists stay aligned across the mixed `.jpg/.png` source directory. **If you add data with other naming conventions, verify the alignment first** — a silent misalignment here produces garbage supervision with no error.
- `filter_files()` drops any pair where `img.size != gt.size`.
- Steps per epoch = `min(len(source_loader), len(target_loader)) = min(⌈4447/16⌉, ⌈4040/16⌉) = 253`.
- In round 2 the source set grows past 4447 while the target set stays at 4040, so `total_step` stays 253 and only a subset of source images is seen each epoch (the loader is shuffled, so coverage is stochastic across epochs).

4. Setup

4.1 Python environment

`uv.lock` is **missing packages the code imports**. One of them is train-blocking:

Package	Needed by	Blocks
<code>scipy</code>	<code>Src/model/SINet/Search Attention.py:6</code> (<code>scipy.stats</code>)	training — SINet cannot be constructed without it
<code>opencv-python</code>	<code>CLS.py</code> , <code>MyTest.py</code> , <code>Eval/MyEval.py</code>	training (CLS)
<code>scikit-learn</code>	<code>Eval/metrics.py:10</code>	evaluation only
<code>prettytable</code>	<code>Eval/MyEval.py:4</code>	evaluation only
<code>tqdm</code>	<code>Eval/MyEval.py:7</code>	evaluation only

```
cd /home/ai-server/Public/lab/Diffusion_Inpaint/S2R-COD
```

```
# add the missing deps, then sync
```

```
uv add scipy opencv-python scikit-learn prettytable tqdm
```

```
uv sync
```

```
# verify
```

```
uv run python -c "
```

```
import torch, torchvision, cv2, sklearn, prettytable, tqdm, PIL, numpy
```

```
print('torch', torch.__version__, '| cuda', torch.cuda.is_available(),
```

```
      '| devices', torch.cuda.device_count())
```

```
print('capability', torch.cuda.get_device_capability(0))
```

```
"
```

Expected on this box: 2× RTX PRO 6000 Blackwell (96 GB each, sm_120), driver 595.84 / CUDA 13.2. [pyproject.toml](https://pypi.org/project/pyproject.toml) already pins the [pytorch-cu128](https://pypi.org/project/pytorch-cu128) index, which is the correct wheel line for Blackwell.

The paper's environment was Python 3.10 / torch 2.0.1. This box needs torch ≥ 2.7 for sm_120, so exact bit-for-bit reproduction is not possible; expect small numerical drift from cuDNN/kernel differences.

4.2 Backbone weights (required — currently missing)

`SINet_ResNet50.initialize_weights()` loads a hard-coded path ([Src/model/SINet/SINet.py:211](#)). The file is not in the repo. It is exactly torchvision's [IMAGENET1K_V2](#) ResNet-50 checkpoint, so you can skip the Google Drive link:

```
curl -L -o Src/model/SINet/resnet50-11ad3fa6.pth \
```

```
https://download.pytorch.org/models/resnet50-11ad3fa6.pth
```

```
# ~102 MB
```

Only if you also want the SINet-v2 row ([Res2Net_v1b.py:195](#)):

```
curl -L -o Src/model/SINetV2/res2net50_v1b_26w_4s-3cf99910.pth \
```

```
https://shanghuagao.oss-cn-beijing.aliyuncs.com/res2net/res2net50\_v1b\_26w\_4s-3cf99910.pth
```

4.3 Extract the datasets

```
cd Dataset
```

```
unrar x Target/Target.rar . # -> Dataset/Target/Image/
```

```
unrar x Test/Test.rar . # -> Dataset/Test/{Image,GT}/
```

```
cd ..
```

```
# verify counts
```

```
ls Dataset/Target/Image | wc -l # 4040
```

```
ls Dataset/Test/Image | wc -l # 2026
```

```
ls Dataset/Test/GT | wc -l # 2026
```

Leaving the `.rar` files in place is harmless — the loaders filter on `.jpg/.png`.

5. The training command

```
cd /home/ai-server/Public/lab/Diffusion_Inpaint/S2R-COD
```

```
uv run python MyTrain.py \
```

```
--network SINet \
```

```
--task S2C \
```

```
--gpu 0 \
```

```
--iteration 2 \
```

```
--save_model ./Snapshot/SINet/S2C/ \
```

```
--source_root ./Dataset/Source/HKU-IS/ \
```

```
--target_root ./Dataset/Target/ \
```

```
--val_root ./Dataset/Val/CAMO/ \
```

```
2>&1 | tee train_sinet_s2c.log
```

That is the whole thing — **one command runs both CSRDA rounds and the CLS step between them**. Do not run round 2 manually.

Non-negotiable details

- **Every path argument must end with /**. The code does raw string concatenation (`opt.save_model + 'Tea_%d.pth'`, `opt.source_root + 'Image/'`). A missing slash silently writes `./Snapshot/SINet/S2CTea_40.pth`.
- **Do not change `--epoch`**. `CLS.py` looks for a checkpoint literally named `Stu_40.pth` ([CLS.py:42](#)); it only exists because `--epoch 40` + the save rule at [MyTrain.py:84-86](#) produce it. Any other value breaks the cycle with `FileNotFoundError`.
- **Do not pass `--alpha/--u/--tau/--a/--b/--c`**. `--task S2C` overwrites them *after* parsing ([MyTrain.py:151-159](#)); anything you pass is discarded. This is by design and already matches the paper.
- Seed is fixed at 42 ([MyTrain.py:163](#)). Note that `torch.backends.cudnn.deterministic` is **not** set, so runs are not bit-reproducible.

For the SINet-v2 row

```
uv run python MyTrain.py --network SINet-v2 --task S2C --gpu 1 --iteration 2 \
```

```
--save_model ./Snapshot/SINet-v2/S2C/ --source_root ./Dataset/Source/HKU-IS/
```

`--network SINet-v2` self-overrides to 100 epochs / batch 32 / decay@50 and enables gradient clipping ([MyTrain.py:174-180](#)). ⚠ See §8 item 3 — `structure_loss` has a bug that will likely raise on modern PyTorch; smoke-test the SINet-v2 path before committing to a long run.

The two networks are independent — run them concurrently on GPU 0 and GPU 1.

6. Inference and evaluation

Use the **teacher** checkpoint from the **final** round.

1. generate masks on COD10K-test (MyTest.py hard-codes ./Dataset/Test/{Image,GT}/)

```
uv run python MyTest.py \
```

```
--network SINet \
```

```
--model_path ./Snapshot/SINet/S2C/Tea_epoch_best.pth \
```

```
--test_save ./Result/SINet/S2C
```

```
ls Result/SINet/S2C | wc -l # must be 2026
```

2. evaluate — MUST run from inside Eval/ (it does `import metrics`)

```
cd Eval
```

```
uv run python MyEval.py \
```

```
--gt_root ../Dataset/Test \
```

```
--pred_root ../Result/SINet/S2C \
```

```
--txt_name SINet/S2C
```

```
cd ..
```

```
cat Eval/eval_txt/SINet/S2C/_eval.txt
```

Leave `--data_lst` and `--model_lst` at their defaults (`[]`). They are declared `type=list`, so passing a string on the CLI explodes it into single characters. With the empty-string defaults, `os.path.join` resolves to the flat `Dataset/Test/GT` ↔ `Result/SINet/S2C` layout, which is what you have.

7. Checklist

Run [preflight.py](#) to verify every item below automatically:

```
uv run python preflight.py          # SINet / S2C defaults
```

```
uv run python preflight.py --network SINet-v2
```

```
uv run python preflight.py --skip-smoke    # skip the GPU fwd/bwd (no VRAM used)
```

```
uv run python preflight.py --deep         # + fully decode every source image
```

It exits 0 only when nothing is blocking, and prints the exact training command on success. It covers packages, CUDA/arch, backbone weights (including SINet's key remap), dataset counts, sorted-order image↔GT pairing, `filter_files` size drops, `MyEval` name reachability, trailing slashes, stale artifacts, disk, the `--epoch↔Stu_40.pth` coupling, the S2C hyperparameter override, `F.upsample` availability, and a real batch + forward/backward at batch 16 / 352².

Pre-flight (manual equivalent)

- ☐ `uv add opencv-python scikit-learn prettytable tqdm && uv sync` completes
- ☐ `uv run python -c "import torch;print(torch.cuda.is_available())"` → `True`
- ☐ `Src/model/SINet/resnet50-11ad3fa6.pth` exists (~102 MB)
- ☐ `Dataset/Target/Image/` has 4040 files
- ☐ `Dataset/Test/Image/` and `Dataset/Test/GT/` have 2026 files each
- ☐ `Dataset/Source/HKU-IS/{Image,GT}/` have 4447 files each
- ☐ `Dataset/Val/CAMO/{Imgs,GT}/` have 250 files each
- ☐ No stale `Dataset/Source/HKU-IS_iteration2/` from a previous attempt (CLS deletes it, but a partial one from a crashed run is confusing)
- ☐ No stale `Snapshot/SINet/S2C/` — **round 2 overwrites `Tea_epoch_best.pth`**; start clean or you cannot tell which round a checkpoint came from
- ☐ All `--*_root / --save_model` arguments end in `/`
- ☐ ≥ 20 GB free disk (checkpoints ≈ 220 MB each × ~15 saved, plus a 113 MB source copy; 1.6 TB free here — fine)

During the run — what a healthy log looks like

- ☐ `[Source Loader] Loaded 4447 image-mask pairs / [Target Loader] Loaded 4040 images`
- ☐ `[INFO] initialize weights from resnet50` printed **twice** per round (student + teacher)
- ☐ `Global Step: XXXX/0253` — 253 steps per epoch

- ☐ `Loss_sup` falls steadily; `Loss_con` starts near 0 (teacher $\equiv$ student at init) and rises, then settles
- ☐ From epoch 21: `Epoch: N, MAE: ..., bestteaMAE: ...` lines appear
- ☐ Round 1 ends $\rightarrow$ `[Info] Average edge loss: ...` $\rightarrow$ a stream of `[PGT] ...` lines. **Sanity-check the kept count:** `ls Dataset/Source/HKU-IS_iteration2/GT | wc -l` should be `4447 + N` with `N` a substantial fraction of 4040. `N  $\approx$  0` or `N  $\approx$  4040` means μ/r are not biting and the cycle is a no-op.
- ☐ Round 2 header `==== Iteration 2/2 started =====`, source loader now reports `4447 + N` pairs

Post-run

- ☐ `Snapshot/SINet/S2C/Tea_epoch_best.pth` exists and is from round 2
- ☐ `Result/SINet/S2C/` contains 2026 PNGs
- ☐ `MyEval.py` prints no `not matching to the ground-truth` warning
- ☐ Compare against the Table 1 targets in §1. Expect ± 0.005 – 0.01 drift from the published values given the different CUDA/PyTorch stack and non-deterministic cuDNN.

Optional but recommended — build the full Table 1 column

To get the Source-Only and Mean Teacher rows too:

- **Source-Only:** train with only `L_sup` (comment out `loss_con` at [MyTrain.py:40.57](#)), `--iteration 1`.
- **Mean Teacher:** `--iteration 1` and swap `ES_Loss` for a plain MSE/BCE consistency. Both are ablations, not the headline result — do them only if you need the deltas.

8. Known deviations and gotchas

These are **all in the original upstream code**. The defaults below are what produced the published numbers, so *for reproduction, change nothing*. They are listed so you are not surprised, and so you know what to fix if you later build on this.

1. 39 epochs, not 40. [MyTrain.py:222](#) is `for epoch_iter in range(1, opt.epoch)  $\rightarrow$  epoch_iter  $\in$  [1, 39]`. The checkpoint named `Tea_40.pth` is saved at `epoch_iter=39` because saving uses `epoch+1`. Leave it alone — `CLS.py` depends on that filename.

2. The LR schedule compounds. [tool.py:36-39](#) does `param_group['lr'] *= decay` where `decay = 0.1 ** (epoch // 30)`. For epochs 1–29 that is $\times 1$ (no-op). For epochs

30–39 it is $\times 0.1$ **every epoch**, so the LR goes $1e-5 \rightarrow 1e-6 \rightarrow \dots \rightarrow 1e-14$. The paper says "dividing it by 10 after 30 epochs", i.e. a flat $1e-5$.

Practical consequence: training is effectively frozen after epoch ~31, and `Tea_epoch_best.pth` is in practice selected from epochs 21–31.

Leave as-is for reproduction. If you want the paper-as-written schedule:

```
# Src/utils/tool.py
```

```
def adjust_lr(optimizer, epoch, decay_rate=0.1, decay_epoch=30, init_lr=1e-4):
```

```
    lr = init_lr * (decay_rate ** (epoch // decay_epoch))
```

```
    for param_group in optimizer.param_groups:
```

```
        param_group['lr'] = lr
```

3. `structure_loss` passes the deprecated `reduce='none'` (SINet-v2 path only).

[tool.py:9](#) — `reduce` is the legacy argument and expects a bool, not a string. **Verified working on torch 2.11.0+cu128** ([preflight.py](#) section I exercises it and it returns a finite scalar), so this is cosmetic here, not a blocker. It is still wrong — the intended spelling is `reduction='none'` — and it may break on a future release, so re-run the preflight if you change the torch version.

4. CLS copies the source tree twice. [CLS.py:12-28](#) — `MyTrain.py` passes `source_root` as *both* `source_root` and `gt_root` ([MyTrain.py:234](#)), so `source_copy_root == gt_copy_root`. The function copies, then sees the destination "already exists", deletes it, and copies again. Functionally correct, just ~113 MB of wasted I/O. Harmless.

5. Round 2 overwrites round 1's `Tea_epoch_best.pth`. Same `--save_model` directory both rounds, and `best_teamae` resets to 1 at [MyTrain.py:197](#). If you want round 1's model (the "Mean Teacher"-ish baseline), copy it out during the CLS step.

6. EMA covers parameters only, not buffers. [tool.py:41-43](#) iterates `.parameters()`. The teacher's BatchNorm running statistics are therefore *not* EMA'd from the student — they are updated by the teacher's own forward passes, which happen in `train()` mode ([MyTrain.py:20](#)) on weakly-augmented target images. This is arguably a feature (target-domain BN statistics) but it is not standard Mean Teacher.

7. Model selection uses CAMO, not COD10K. [MyTrain.py:230](#) validates on `Dataset/Val/CAMO/`. COD10K-test is never touched during training — good, no test leakage

— but the selected checkpoint is the CAMO-MAE-optimal one, not the COD10K-optimal one. Keep it that way; changing it invalidates the comparison.

8. Validation only runs after epoch 20. Before that no `Tea_epoch_best.pth` exists. If you kill a run early, `CLS.py` falls back to `Tea_40.pth` ([CLS.py:61-63](#)) — which also will not exist. Let each round finish.

9. Resource and time estimates

GPU memory	~10–14 GB at batch 16 / 352^2 (student fwd+bwd + teacher fwd). 96 GB available.
Steps	253 / epoch × 39 epochs × 2 rounds = 19,734 optimizer steps
CLS overhead	2 passes × 4040 images × 2 models = ~16k forwards per cycle, a few minutes
Wall clock (SINet, 1× RTX PRO 6000)	rough estimate ~2–4 h total for both rounds. Time the first epoch and extrapolate rather than trusting this.
Disk	~3.5 GB (checkpoints) + 113 MB (source copy) + ~1.7 GB (extracted data)
Parallelism	SINet on <code>--gpu 0</code> and SINet-v2 on <code>--gpu 1</code> can run simultaneously

Note `num_workers=6` is hard-coded in the loader calls ([MyTrain.py:204](#), [:208](#)) — raise it there if the GPU is data-starved.

10. Quick reference — full sequence

```
cd /home/ai-server/Public/lab/Diffusion_Inpaint/S2R-COD
```

```
# --- setup (once) ---
```

```
uv add opencv-python scikit-learn prettytable tqdm && uv sync

curl -L -o Src/model/SINet/resnet50-11ad3fa6.pth \

https://download.pytorch.org/models/resnet50-11ad3fa6.pth

cd Dataset && unrar x Target/Target.rar . && unrar x Test/Test.rar . && cd ..

# --- train (both CSRDA rounds + CLS, one command) ---

uv run python MyTrain.py --network SINet --task S2C --gpu 0 --iteration 2 \

--save_model ./Snapshot/SINet/S2C/ --source_root ./Dataset/Source/HKU-IS/ \

--target_root ./Dataset/Target/ --val_root ./Dataset/Val/CAMO/ \

2>&1 | tee train_sinet_s2c.log

# --- test ---

uv run python MyTest.py --network SINet \

--model_path ./Snapshot/SINet/S2C/Tea_epoch_best.pth \

--test_save ./Result/SINet/S2C

# --- evaluate ---

cd Eval && uv run python MyEval.py --gt_root ../Dataset/Test \

--pred_root ../Result/SINet/S2C --txt_name SINet/S2C && cd ..
```